Aanchal wadhwa
T2- T23
2401125

EXPERIMENT NO.9

Lab outcomes :
LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

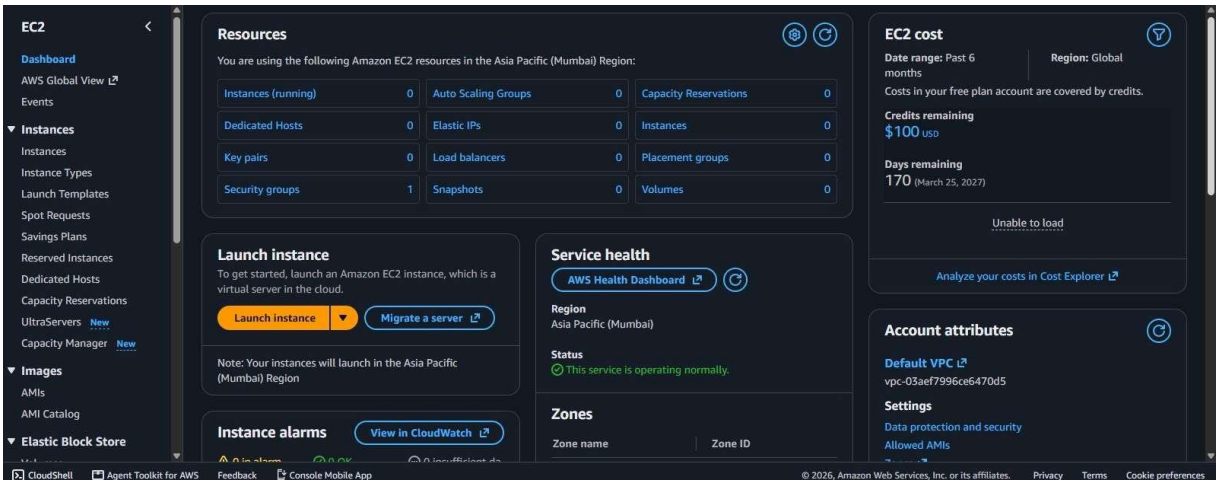
Theory:

Step 1: Create or verify your AWS account 1.

Open the AWS Management Console .

2. Sign in to your AWS account.

3. Select the Mumbai region (ap-south-1).
4.
5. Open the EC2 Dashboard and check that your account is ready to launch instances.



The account must have permissions to create EC2 instances and use the required

networking resources. Step 2: Install Terraform on Windows

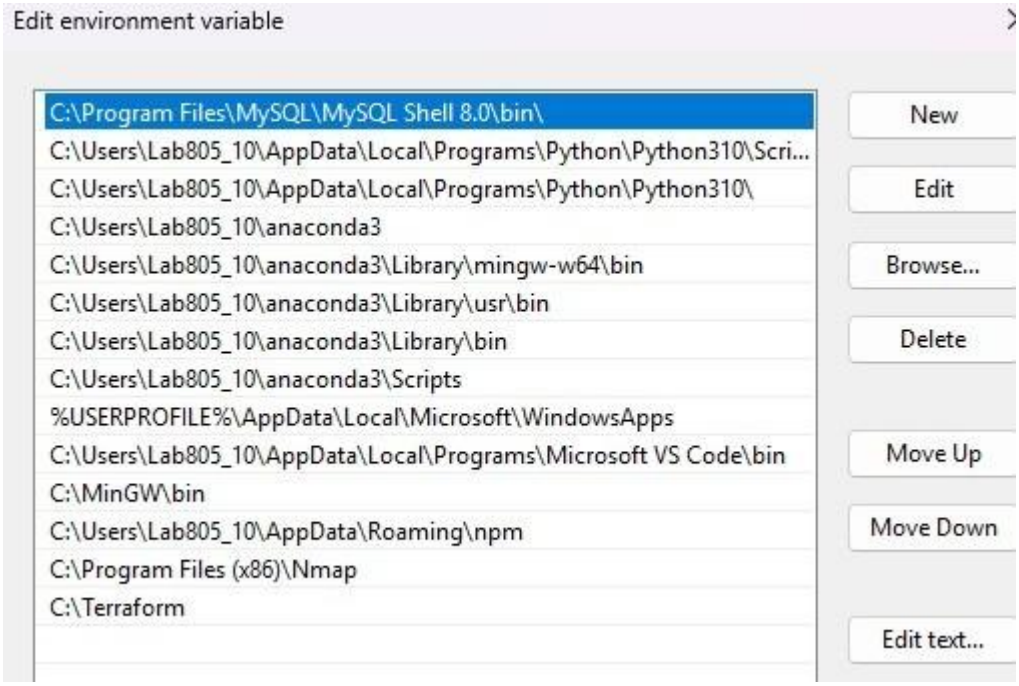
1. Visit the official Terraform installation page .

2. Download the Windows AMD64 ZIP package.



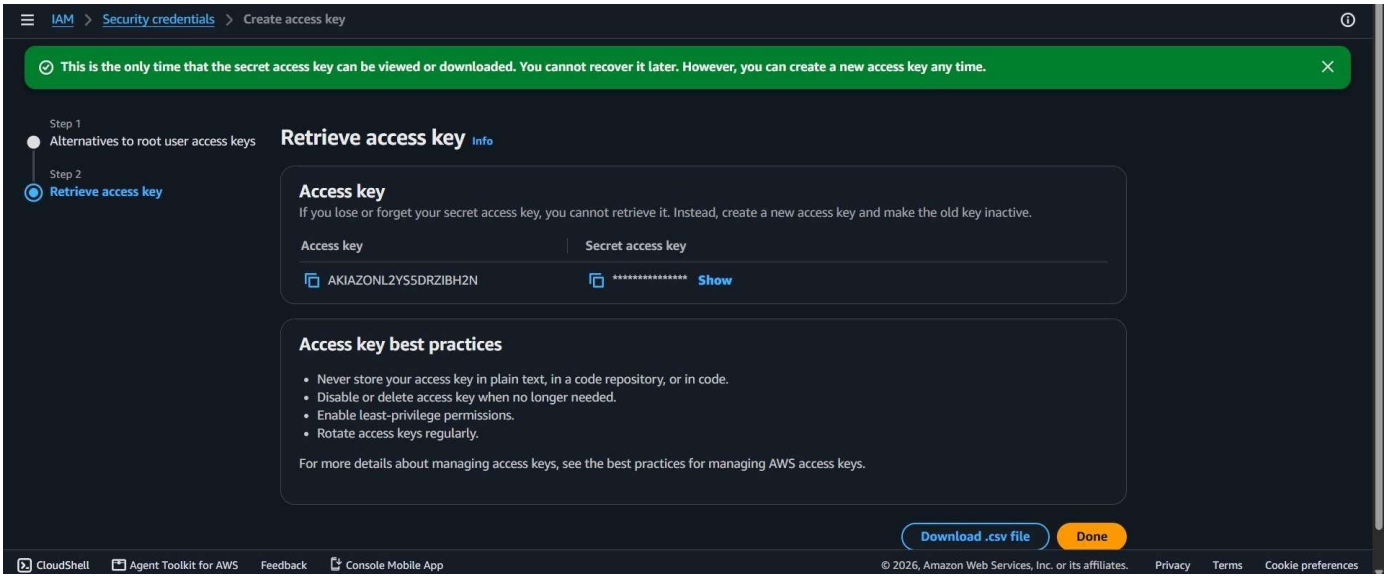
3. Extract terraform.exe into a folder, for example: C:\Terraform

4. Add C:\Terraform to the Windows PATH environment variable.



5. Open a new Command Prompt or PowerShell window.





```
PS D:\terraform_1.16.5_windows_386> dir


    Directory: D:\terraform_1.16.5_windows_386


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         10/8/2026  11:10 PM           4950 LICENSE.txt
-a----         10/8/2026  11:10 PM      116640904 terraform.exe


PS D:\terraform_1.16.5_windows_386> notepad main.tf
```

main.tf lec1 web dev.txt

File Edit View

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "demo" {
  ami           = "ami-0c02fb55956c7d316"
  instance_type = "t2.micro"

  tags = {
    Name = "Terraform-Demo"
  }
}
```

```
PS D:\terraform_1.16.5_windows_386> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.68.0...
- Installed hashicorp/aws v6.68.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS D:\terraform_1.16.5_windows_386> terraform apply
```

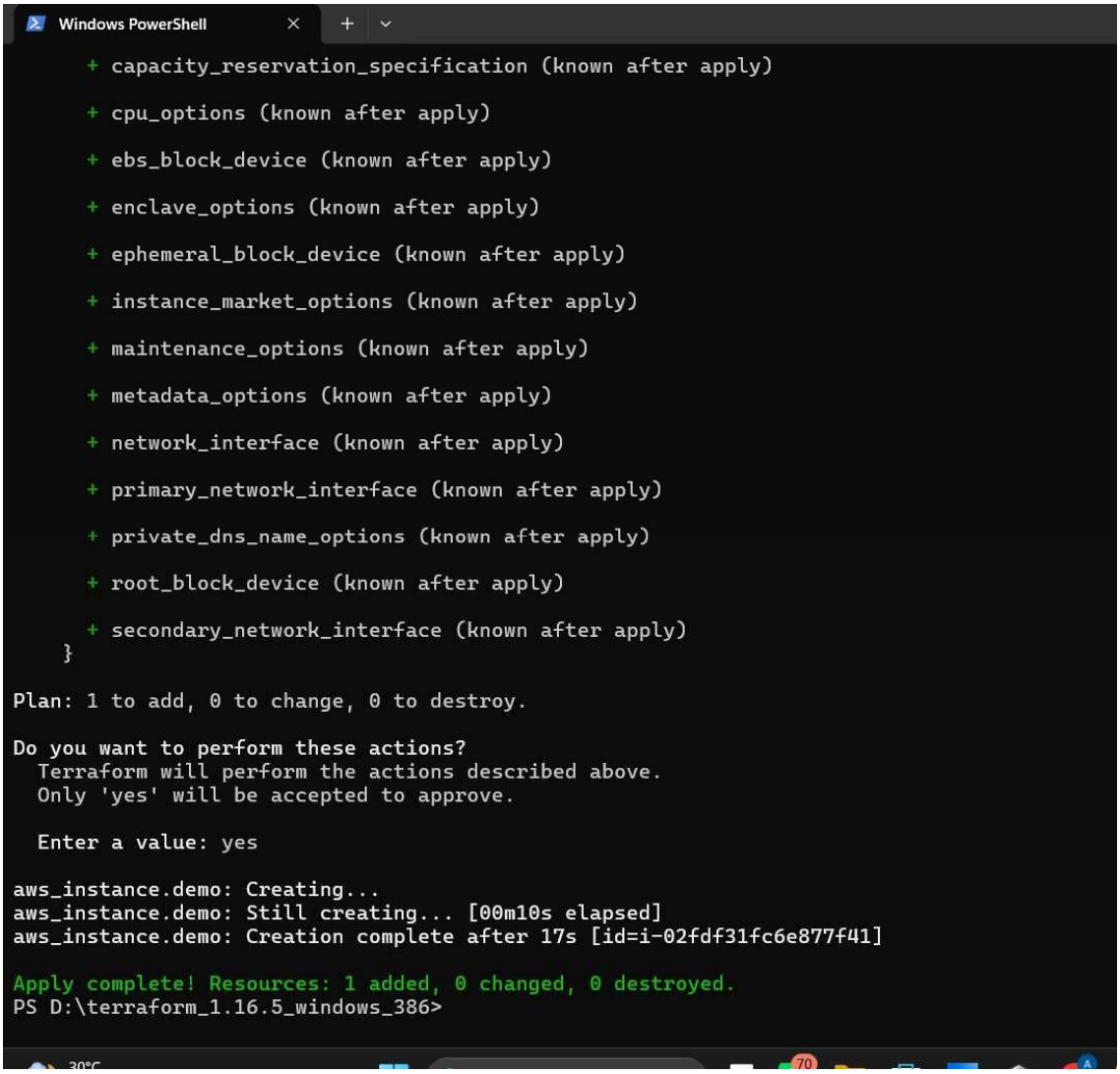
```
PS D:\terraform_1.16.5_windows_386> terraform validate
Success! The configuration is valid.

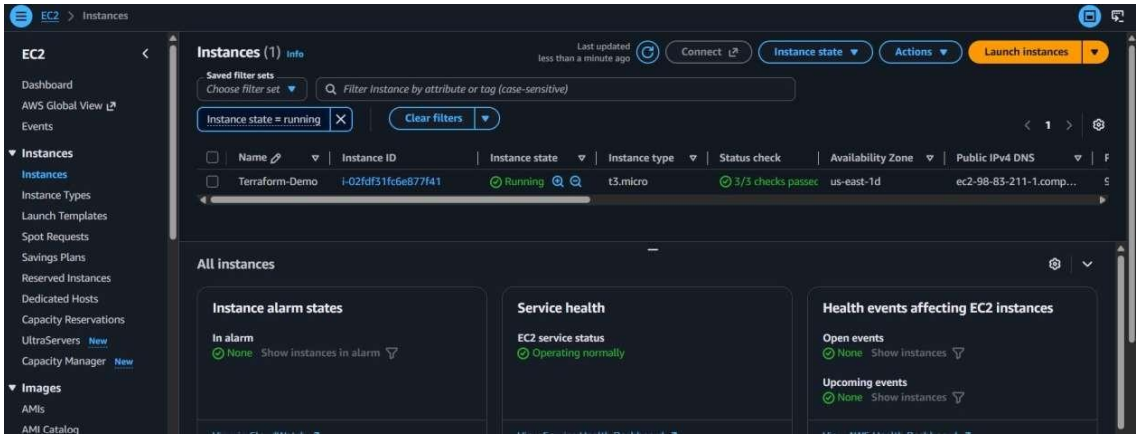
PS D:\terraform_1.16.5_windows_386> terraform plan

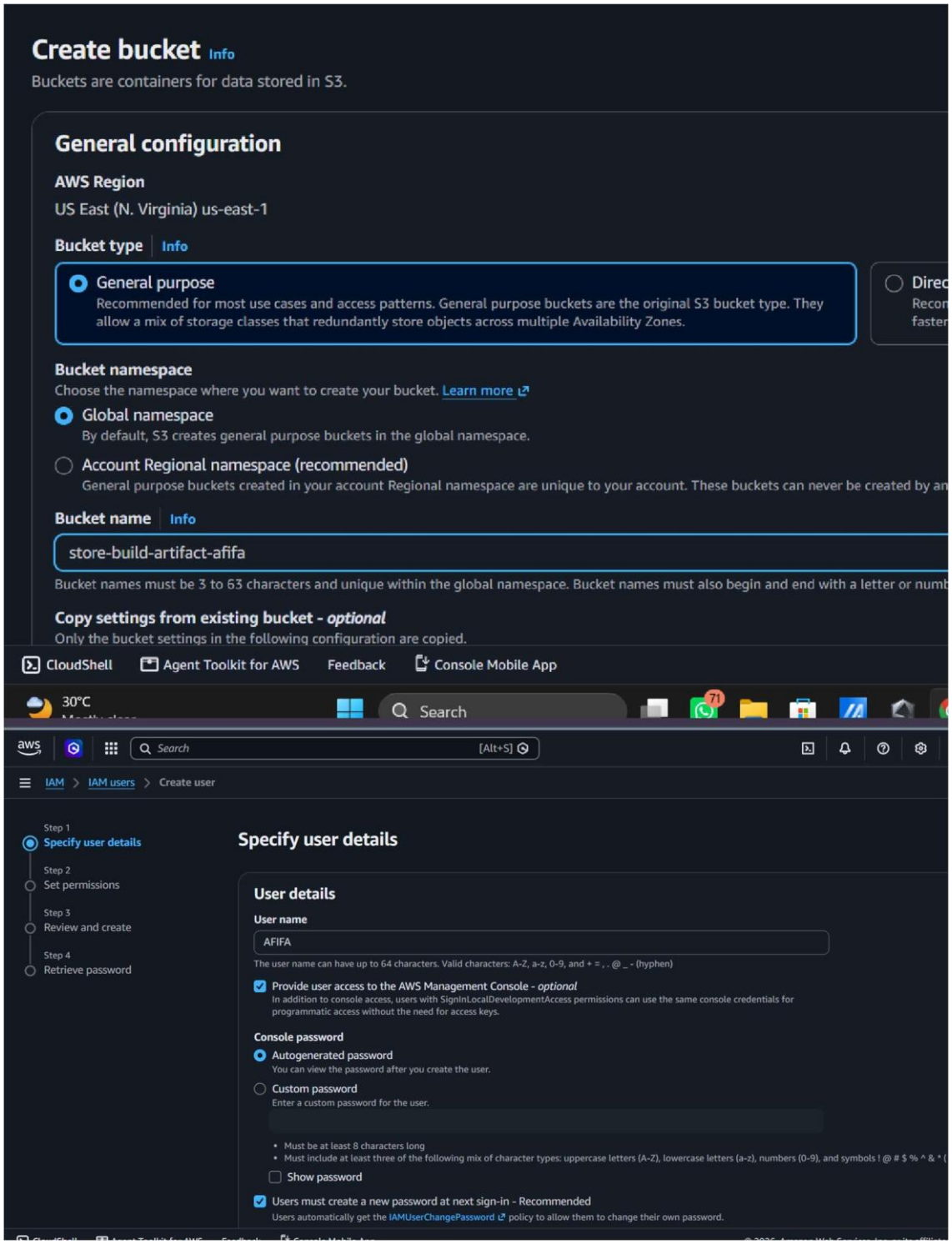
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

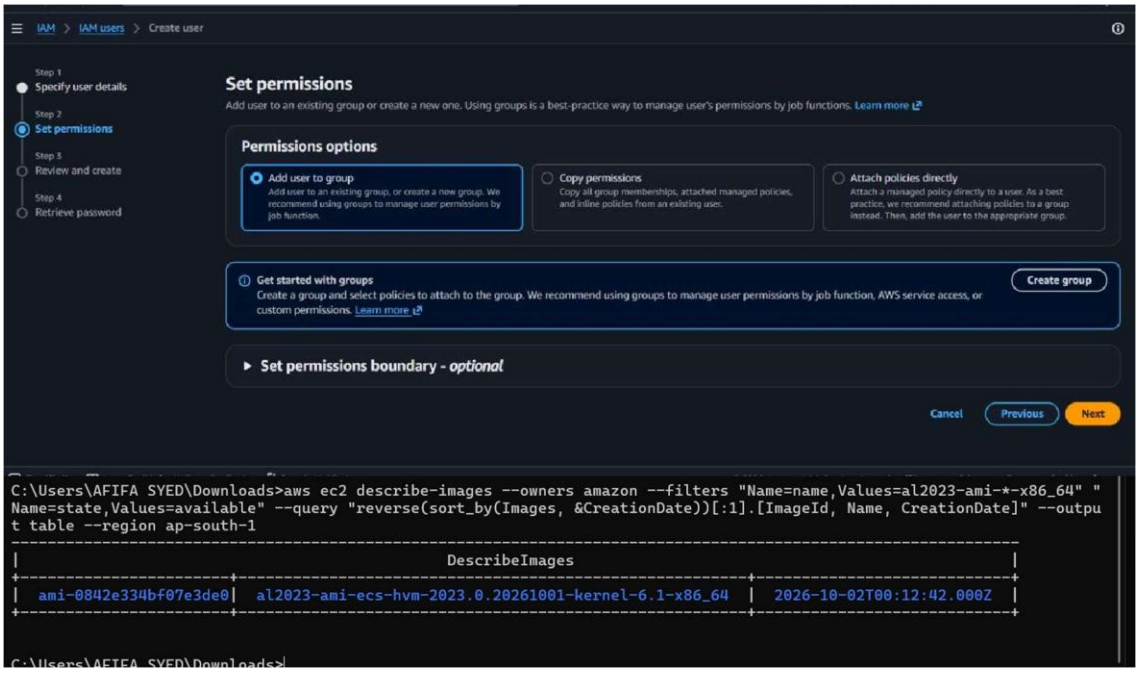
Terraform will perform the following actions:

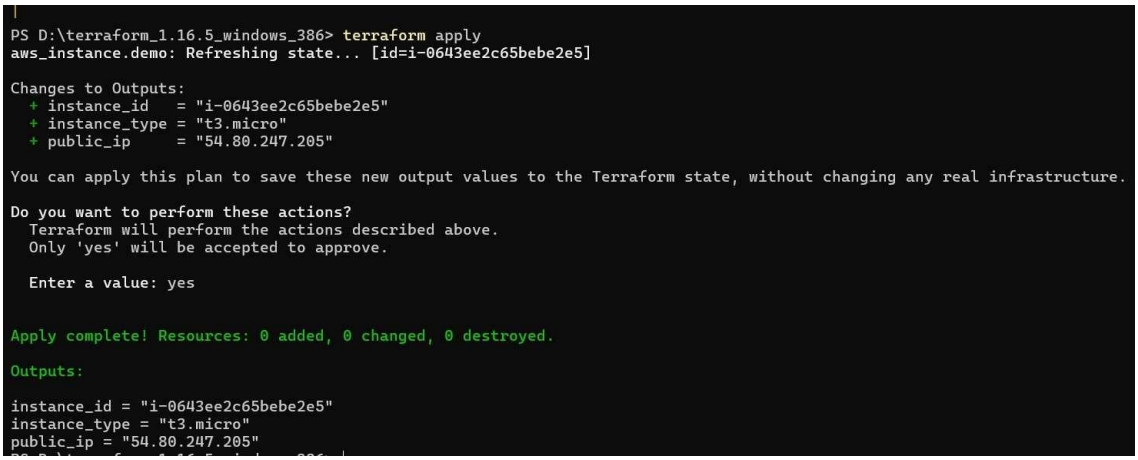
  # aws_instance.demo will be created
  + resource "aws_instance" "demo" {
      + ami                                  = "ami-0c02fb55956c7d316"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
```


Windows PowerShell
+ capacity_reservation_specification (known after apply)
+ cpu_options (known after apply)
+ ebs_block_device (known after apply)
+ enclave_options (known after apply)
+ ephemeral_block_device (known after apply)
+ instance_market_options (known after apply)
+ maintenance_options (known after apply)
+ metadata_options (known after apply)
+ network_interface (known after apply)
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_instance.demo: Creating...
aws_instance.demo: Still creating... [00m10s elapsed]
aws_instance.demo: Creation complete after 17s [id=i-02fdf31fc6e877f41]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS D:\terraform_1.16.5_windows_386>


EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
Instances (1) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Terraform-Demo
i-02fdf31fc6e877f41
Running
t3.micro
3/3 checks passed
us-east-1d
ec2-98-83-211-1.comp...
All instances
Instance alarm states
In alarm
None
Show instances in alarm
Service health
EC2 service status
Operating normally
Health events affecting EC2 instances
Open events
None
Show instances
Upcoming events
None
Show instances


Create bucket Info
Buckets are containers for data stored in S3.
General configuration
AWS Region
US East (N. Virginia) us-east-1
Bucket type Info
General purpose
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.
Bucket namespace
Choose the namespace where you want to create your bucket. Learn more
Global namespace
By default, S3 creates general purpose buckets in the global namespace.
Account Regional namespace (recommended)
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by an
Bucket name Info
store-build-artifact-afifa
Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or numb
Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
30°C
Search
Search
[Alt+S]
IAM
IAM users
Create user
Step 1
Specify user details
Step 2
Set permissions
Step 3
Review and create
Step 4
Retrieve password
Specify user details
User details
User name
AFIFA
The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)
Provide user access to the AWS Management Console - optional
In addition to console access, users with SignInLocalDevelopmentAccess permissions can use the same console credentials for programmatic access without the need for access keys.
Console password
Autogenerated password
You can view the password after you create the user.
Custom password
Enter a custom password for the user.
Must be at least 8 characters long
Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # $ % ^ & *
Show password
Users must create a new password at next sign-in - Recommended
Users automatically get the IAMUserChangePassword policy to allow them to change their own password.





.tf files in the same folder are treated as one Terraform configuration. The filename does not determine what gets executed. Terraform destroy can be used to withdraw access from the infrastructure. Student must do this.

Output: Clear and visible screenshots must be attached.

12. Learning Outcomes Achieved :

Students gained practical knowledge of using Terraform to define, provision, and manage AWS cloud infrastructure through Infrastructure as Code (IaC).

13. Conclusion:

    1. Applications of the studied technique in industry

       Terraform enables automated and consistent provisioning of cloud infrastructure, supporting efficient deployment and management of AWS resources.

14. Engineering Relevance

    The experiment demonstrates Infrastructure as Code practices that improve scalability, reliability, repeatability, and efficiency in cloud-based engineering.

15. Skills Developed

    Students developed skills in writing Terraform configurations, executing Terraform commands, and provisioning AWS resources using Terraform.